3296. Minimum Number of Seconds to Make Mountain Height Zero

Solved

Medium | Topics | Companies | Hint

You are given an integer `mountainHeight` denoting the height of a mountain.

You are also given an integer array `workerTimes` representing the work time of workers in **seconds**.

The workers work **simultaneously** to **reduce** the height of the mountain. For worker `i`:

- To decrease the mountain's height by `x`, it takes `workerTimes[i] + workerTimes[i] * 2 + ... + workerTimes[i] * x` seconds. For example:
  - To reduce the height of the mountain by 1, it takes `workerTimes[i]` seconds.
  - To reduce the height of the mountain by 2, it takes `workerTimes[i] + workerTimes[i] * 2` seconds, and so on.

Return an integer representing the **minimum** number of seconds required for the workers to make the height of the mountain 0.

Example 1:

> **Input:** `mountainHeight = 4, workerTimes = [2,1,1]`
>
> **Output:** 3
>
> **Explanation:**
>
> One way the height of the mountain can be reduced to 0 is:
>
> - Worker 0 reduces the height by 1, taking `workerTimes[0] = 2` seconds.
> - Worker 1 reduces the height by 2, taking `workerTimes[1] + workerTimes[1] * 2 = 3` seconds.
> - Worker 2 reduces the height by 1, taking `workerTimes[2] = 1` second.
>
> Since they work simultaneously, the minimum time needed is `max(2, 3, 1) = 3` seconds.

Example 2:

> **Input:** `mountainHeight = 10, workerTimes = [3,2,2,4]`
>
> **Output:** 12
>
> **Explanation:**
>
> - Worker 0 reduces the height by 2, taking `workerTimes[0] + workerTimes[0] * 2 = 9` seconds.
> - Worker 1 reduces the height by 3, taking `workerTimes[1] + workerTimes[1] * 2 + workerTimes[1] * 3 = 12` seconds.
> - Worker 2 reduces the height by 3, taking `workerTimes[2] + workerTimes[2] * 2 + workerTimes[2] * 3 = 12` seconds.
> - Worker 3 reduces the height by 2, taking `workerTimes[3] + workerTimes[3] * 2 = 12` seconds.
>
> The number of seconds needed is `max(9, 12, 12, 12) = 12` seconds.

Example 3:

> **Input:** `mountainHeight = 5, workerTimes = [1]`
>
> **Output:** 15
>
> **Explanation:**
>
> There is only one worker in this example, so the answer is `workerTimes[0] + workerTimes[0] * 2 + workerTimes[0] * 3 + workerTimes[0] * 4 + workerTimes[0] * 5 = 15`.

Constraints:

- $1 \le mountainHeight \le 10^5$
- $1 \le workerTimes.length \le 10^4$
- $1 \le workerTimes[i] \le 10^6$

Python:

```
import math

class Solution:
    def minNumberOfSeconds(self, mountainHeight: int, workerTimes: List[int]) -> int:
        if mountainHeight == 0: return 0

        # 1. Theoretical Seed: Using the global capacity S = sum(1/sqrt(w))
        # This is an "Academic Pivot"—modeling the global system.
        inv_sqrt_sum = sum(1.0 / math.sqrt(w) for w in workerTimes)
        theoretical_t = 0.5 * (mountainHeight / inv_sqrt_sum)**2

        # 2. Tightened Search Range
        # Instead of 0 to 1e18, we use a window based on the model.
        # We add a buffer to account for the discrete steps.
        low = int(theoretical_t * 0.5)
        high = int(theoretical_t * 2.0) + max(workerTimes)
```

```
        # 3. Discrete Global Function
        def get_total_h(T):
            total = 0
            for w in workerTimes:
                # Solving the quadratic for each worker: w*x(x+1)/2 <= T
                total += int((math.sqrt(1 + 8 * T / w) - 1) / 2)
                if total >= mountainHeight: return True
            return total >= mountainHeight

        # 4. Final Convergence
        ans = high
        while low <= high:
            mid = (low + high) // 2
            if get_total_h(mid):
                ans = mid
                high = mid - 1
            else:
                low = mid + 1
        return ans
```

JavaScript:

```
/*************************😎*************************/
function minNumberOfSeconds(e,t){let o=0,r=1e18,n=r;for(;o<=r;){let
f=Math.floor(o+(r-o)/2);canComplete(f,e,t)?(n=f,r=f-1):o=f+1}return n}function
canComplete(e,t,o){let r=0;for(let n of o){let o=0,f=t;for(;o<=f;){let
t=Math.floor(o+(f-o)/2);n*(t*(t+1))/2<=e?o=t+1:f=t-1}if(r+=f,r>=t)return!0}return r>=t}
```

Java:

```
class Solution {
    public long minNumberOfSeconds(int mountainHeight, int[] workerTimes) {
        long l=1,r=(long)(1e6+1)*(sum(mountainHeight+1));
        // System.out.println(r);
        long ans=-1;
        while(l<=r){
            long mid=(r-l)/2+l;
            if(isValid(mountainHeight,workerTimes,mid)){
                r=mid-1;
                ans=mid;
            }else l=mid+1;
        }
        return ans;
    }
    public boolean isValid(int moutainHeight,int[] workerTimes,long limit){
        long tot=0L;
        for(int i=0;i<workerTimes.length;i++){
```

```
            long t=workerTimes[i];
            long x=solve(t,t,-2*limit);
            tot+=x;
        }
        return tot>=moutainHeight;
    }
    public long sum(long n){
        return (n*(n+1))/2;
    }
    public long solve(long a,long b,long c){
        return (long)(-b+Math.sqrt(b*b-(4.0*a*c)))/(2*a);
    }
}
```